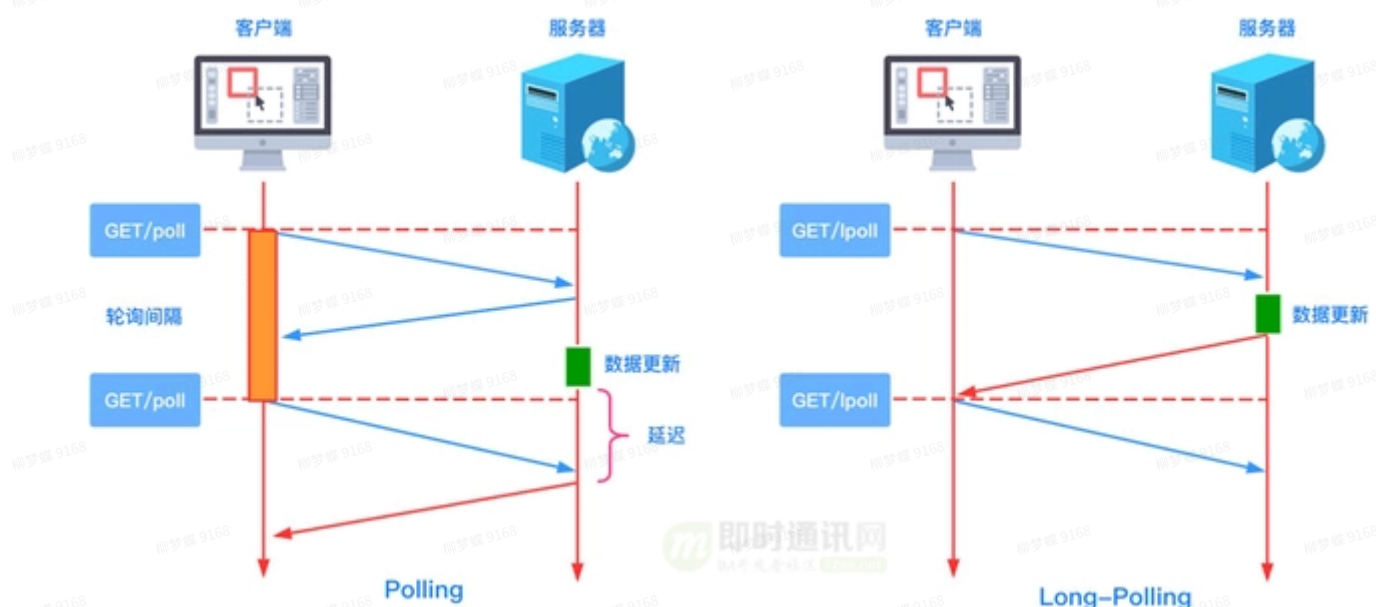


优雅的WebSocket

WebSocket 诞生背景

短轮询

浏览器每隔一段时间向服务器发出 HTTP 请求，然后服务器返回最新的数据给客户端。



comet

当服务器收到客户端发来的请求后，不会直接进行响应，而是先将这个请求挂起，然后判断服务器端数据是否有更新。如果有更新，则进行响应，如果一直没有数据，则到达一定的时间限制(服务器端设置)后关闭连接。

长轮询和短轮询比起来，明显减少了很多不必要的http请求次数，相比之下节约了资源。长轮询的缺点在于，连接挂起也会导致资源的浪费。

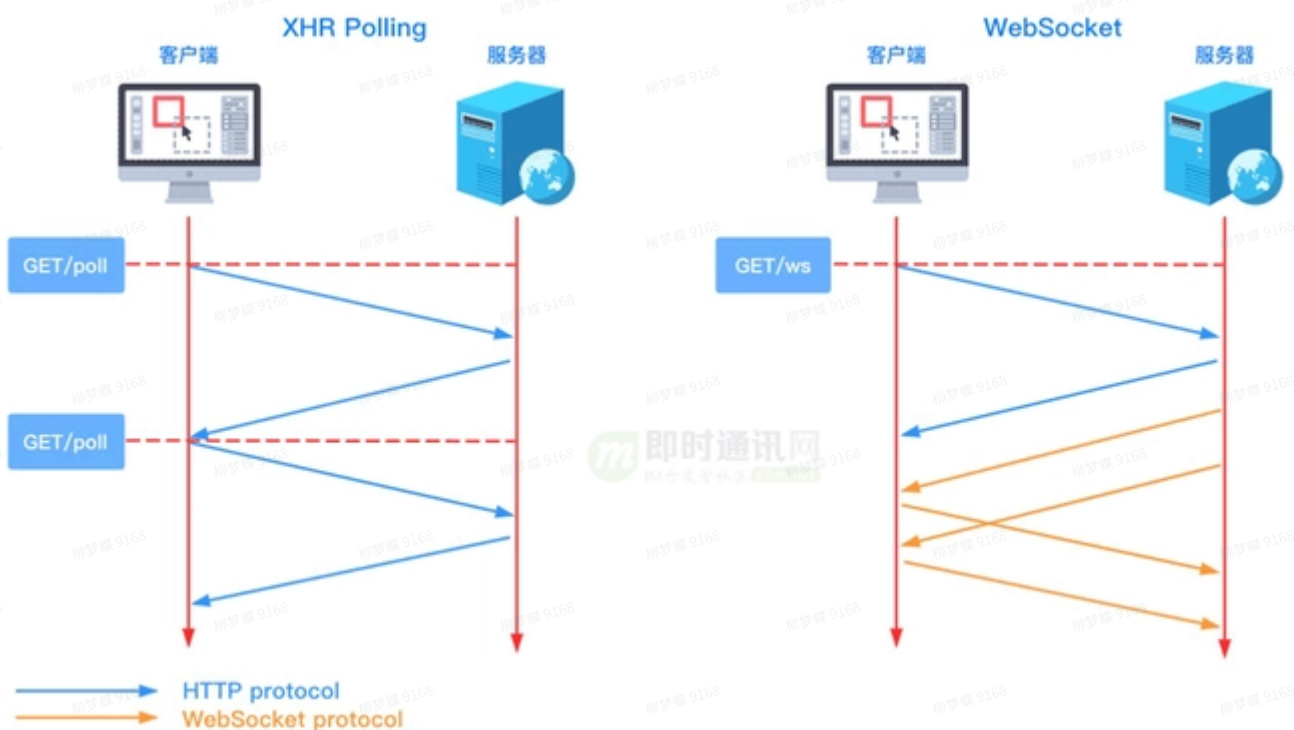
SSE

SSE是HTML5新增的功能，全称为Server-Sent Events。它可以允许服务推送数据到客户端。SSE在本质上就与之前的长轮询、短轮询不同，虽然都是基于http协议的，但是轮询需要客户端先发送请求。而SSE最大的特点就是不需要客户端发送请求，可以实现只要服务器端数据有更新，就可以马上发送到客户端。

SSE的优势很明显，它不需要建立或保持大量的客户端发往服务器端的请求，节约了很多资源，提升应用性能。并且后面会介绍道，SSE的实现非常简单，并且不需要依赖其他插件。

WebSocket

WebSocket是Html5定义的一个新协议，与传统的http协议不同，该协议可以实现服务器与客户端之间全双工通信。简单来说，首先需要在客户端和服务端建立起一个连接，这部分需要http。连接一旦建立，客户端和服务端就处于平等的地位，可以相互发送数据，不存在请求和响应的区别。



如何实现一个优雅的WebSocket

建立最简单的一对一连接

```
1 // web
2 new WebSocket()
3
4 // server
5 new WebSocketServer()
```

加入多对一

```
1 // server
2 const obj = []
3 wss.on('connection', function (ws,user) {
4   const id = user.url.split('/')[1]
5   obj.push({ WS: ws, user: id})
6 })
```

加入心跳

没有消息websocket一分钟后会自动断开

```
1 // web 根据情况发送信息
2 ws.send("ping")
3
4 // server 接收到ping 返回pong
5 ws.on('message', function (message) {
6   const res = { type: id, data: JSON.parse(message)}
7   if(res.data === 'ping'){
8     ws.send(JSON.stringify({ type: id, data: 'pong' } ))
9   }
10 })
11
```

建立聊天室

```
1 // server
2 const obj = []
3 wss.on('connection', function (ws,user) {
4   const id = user.url.split('/')[1]
5   obj.push({ WS: ws, user: id})
6
7   ws.on('message', function (message) {
8     // 可以根据判断来筛选返回消息
9     for(let key of obj){
10       key.WS.send(JSON.stringify(res))
11     }
12
13   });
14 })
```

解决多次重复连接

```
1 // web 监听是否处于焦点
2 window.addEventListener("focus", handleFocus);
3 window.addEventListener("blur", handleBlur);
```

加入域名池

```
1 // web 动态添加WebSocket的地址
2 new WebSocket(`ws://localhost:${port}/${this.user}`)
3
4 // server
5 wss = new WebSocketServer({ port: 8181 });
6 wss = new WebSocketServer({ port: 8182 });
```